5 Controlling your configuration with the Spring Cloud Configuration Server

This chapter covers

- Separating the service configuration from the service code
- Configuring a Spring Cloud Configuration Server
- Integrating a Spring Boot microservice with a configuration server
- Encrypting sensitive properties
- Integrating the Spring Cloud Configuration Server with HashiCorp Vault

Software developers always hear about the importance of keeping the application configuration separate from the code. In most scenarios, this means not using hardcoded values in the code. Forgetting this principle can make changing an application more complicated because every time a change to the configuration is made, the application has to be recompiled and/or redeployed.

Completely separating the configuration information from the application code allows developers and operations to make changes to their configurations without going through a recompile process. But it also introduces complexity, because now developers have another artifact to manage and deploy with the application.

Many developers turn to property files (YAML, JSON, or XML) to store their configuration information. Configuring your application in these files becomes a simple task, so easy that most developers never do more than placing their configuration file under source control (if that) and deploying the file as part of their application. This approach might work with a small number of applications, but it quickly falls apart when dealing with cloud-based applications that can contain hundreds of microservices, where each microservice, in turn, might have multiple service instances running. Suddenly, an easy and straightforward process becomes a big deal, and an entire team has to wrestle with all of the configuration files.

For example, let's imagine we have hundreds of microservices, and each microservice contains different configurations for three environments. If we don't manage those files

outside the application, every time there is a change, we must search for the file in the code repository, follow the integration process (if there is one), and restart the application. To avoid this catastrophic scenario, as a best practice for cloud-based microservices development, we should consider the following:

- Completely separate the configuration of an application from the actual code being deployed.
- Build immutable application images that never change as these are promoted through your environments.
- Inject any application configuration information at server startup through either environment variables or a centralized repository that the microservices read on startup.

This chapter introduces you to the core principles and patterns needed to manage application configuration data in a cloud-based microservice application. We'll then build a configuration server, integrate the server with Spring and with Spring Boot clients, and then learn about protecting our more sensitive configurations.

5.1 On managing configuration (and complexity)

Managing application configuration is critical for microservices running in the cloud because microservice instances need to be launched quickly with minimal human intervention. When a person needs to manually configure or touch a service to get it deployed, this becomes an opportunity for configuration drift, an unexpected outage, and a lag time in responding to scalability challenges within the application. Let's begin our discussion about application configuration management by establishing four principles we want to follow:

- *Segregate*—We need to completely separate the service configuration information from the actual physical deployment of a service. In fact, application configuration shouldn't be deployed with the service instance. Instead, configuration information should either be passed as environment variables to the starting service or read from a centralized repository when the service starts.
- *Abstract*—We also need to abstract access to configuration data behind a service interface. Instead of writing code that directly reads the service repository, whether file-based or a JDBC database, we should use a REST-based JSON service to retrieve the application's configuration data.
- *Centralize*—Because a cloud-based application might literally have hundreds of services, it's critical to minimize the number of different repositories used to hold configuration data. Centralize your application configuration into as few repositories as possible.

- *Harden*—Because your application configuration information is going to be completely segregated from your deployed service and centralized, it's critical that the solution you utilize and implement be highly available and redundant.

One of the key things to remember is that when you separate your configuration information outside of your actual code, you're creating an external dependency that will need to be managed and version controlled. We can't emphasize enough that the application configuration data needs to be tracked and version controlled because mismanaged application configuration is a fertile breeding ground for difficult-to-detect bugs and unplanned outages.

5.1.1 Your configuration management architecture

As you'll remember from the previous chapters, the loading of configuration management for a microservice occurs during the bootstrapping phase of the microservice. As a reminder, figure 5.1 shows the microservice lifecycle.

Figure 5.1 When a microservice starts up, it goes through multiple steps in its lifecycle. The application configuration data is read during the service's bootstrapping phase.

Let's take the four principles we laid out in section 5.1 (segregate, abstract, centralize, and harden) and see how these apply when the service is bootstrapped. Figure 5.2 presents the bootstrapping process in more detail and shows how a configuration service plays a critical role in this step.

Figure 5.2 Configuration management conceptual architecture

In figure 5.2, several activities are taking place. Here's a rundown for each step in the figure:

1. When a microservice instance comes up, it calls a service endpoint to read its configuration information, which is specific to the environment it's operating in. The connection information for the configuration management (connection credentials, service endpoint, and so on) then passes into the microservice as it starts.

2. The actual configuration resides in a repository. Based on the implementation of your configuration repository, you can choose different ways to hold your configuration data. This can include files under source control, relational databases, or key-value data stores.
3. The actual management of the application configuration data occurs independently of how the application is deployed. Changes to configuration management are typically handled through the build and deployment pipeline, where modifications can be tagged with version information and deployed through the different environments (development, staging, production, and so forth).
4. When the configuration management changes, the services that use that application configuration data must be notified of the alteration and refresh their copy of the application data.

At this point, we've worked through the conceptual architecture that illustrates the different pieces of a configuration management pattern and how these pieces fit together. Now we will look at the different solutions to achieve configuration management and then look at a concrete implementation.

1.2 Implementation choices

Fortunately, you can choose from a large number of battle-tested open source projects to implement a configuration management solution. Let's look at several of the different choices available and compare them. Table 5.1 lays out the options.

Table 5.1 Open source projects for implementing a configuration management system

Project name	Description	Characteristics
etcd	Written in Go. Used for service discovery and key-value management. Uses the raft protocol (https://raft.github.io/) for its distributed computing model.	Very fast and scalable Distributable Command-line driven Easy to use and set up
Eureka	Written by Netflix. Extremely battle-tested. Used for both service discovery and key-value management.	Distributed key-value store Flexible but takes some effort to set up Offers dynamic client refresh out of the box
Consul	Written by HashiCorp. Similar to etcd and Eureka but uses a different algorithm for its distributed computing model.	Fast Offers native service discovery with the option to integrate directly with DNS Doesn't offer client dynamic refresh out of the box
Zookeeper	An Apache project. Offers distributed locking capabilities. Often used as a configuration management solution for accessing key-value data.	Oldest, most battle-tested of the solutions Most complex to use Can be used for configuration management, but consider only if you're already using Zookeeper in other pieces of your architecture
Spring Cloud ConfigurationServer	An open source project. Offers a general configuration manage-	Non-distributed key-value store

	ment solution with different backends.	Offers tight integration for Spring and non-Spring services Can use multiple backends for storing configuration data including a shared filesystem, Eureka, Consul, or Git

All the solutions in table 5.1 can easily be used to build a configuration management solution. For the examples in this chapter, and throughout the rest of the book, we'll use the Spring Cloud Configuration Server (often called the Spring Cloud Config Server or, simply, the Config Server), which adapts perfectly to our Spring microservices architecture. We chose this solution because

- The Spring Cloud Configuration Server is easy to set up and use.
- Spring Cloud Config integrates tightly with Spring Boot. You can literally read all of your application's configuration data with a few simple-to-use annotations.
- The Config Server offers multiple backends for storing configuration data.
- Of all of the solutions in table 5.1, the Config Server can integrate directly with the Git source control platform and with the HashiCorp Vault. We will explain these topics later on this chapter.

For the rest of this chapter, we'll do the following:

1. Set up a Spring Cloud Configuration Server. We'll demonstrate three different mechanisms for serving application configuration data—one using the filesystem, another using a Git repository, and another using the HashiCorp Vault.
2. Continue to build the licensing service to retrieve data from a database.
3. Hook the Spring Cloud Config service into your licensing service to serve up the application's configuration data.

5.2 Building our Spring Cloud Configuration Server

The Spring Cloud Configuration Server is a REST-based application that is built on top of Spring Boot. The Config Server doesn't come as a standalone server. Instead, you can choose to either embed it in an existing Spring Boot application or start a new Spring Boot project with the server embedded in it. The best practice is to keep things separated.

The first thing we need to do to build our configuration server is to create a Spring Boot project with Spring Initializr (**https://start.spring.io/**). To achieve this, we'll implement the following steps in the Initializr's form. Once filled in, your Spring Initializr form should look like that shown in figures 5.3 and 5.4. In the form, you'll

1. Select Maven as the project type.
2. Select Java as the language.
3. Select the latest or more stable Spring version.
4. Write `com.optimagrowth` as the group and `configserver` as the artifact.
5. Expand the options list and write.
    1. `Configuration Server` as its name.
    2. `Configuration server` as its description.
    3. `com.optimagrowth.configserver` as its package name.
6. Select JAR Packaging.
7. Select Java 11 as the Java version.
8. Add the Config Server and Spring Boot Actuator dependencies.

Figure 5.3 Spring Initializr configuration for the Spring Configuration Server

Figure 5.4 Spring Initializr Config Server and Spring Boot Actuator dependencies

Once you've created and imported the form as a Maven project into your preferred IDE, you should have a pom.xml file in the root of the config server project directory that looks like the code in the following listing.

Listing 5.1 Maven pom file for the Spring Configuration Server

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-parent</artifactId>
     <version>2.2.4.RELEASE</version>                    (1)
     <relativePath/> <!-- lookup parent from repository -->
```

```
</parent>
<groupId>com.optimagrowth</groupId>
<artifactId>configserver</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>Configuration Server</name>
<description>Configuration Server</description>
<properties>
  <java.version>11</java.version>
  <spring-cloud.version>
     Hoxton.SR1
  </spring-cloud.version>                                    2
</properties>

<dependencies>                                               3
  <dependency>
     <groupId>org.springframework.cloud</groupId>
     <artifactId>spring-cloud-config-server</artifactId>
  </dependency>
  <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-test</artifactId>
     <scope>test</scope>
     <exclusions>
       <exclusion>
          <groupId>org.junit.vintage</groupId>
          <artifactId>
            junit-vintage-engine
          </artifactId>
       </exclusion>
     </exclusions>
  </dependency>
</dependencies>
<dependencyManagement>                                       4
  <dependencies>
     <dependency>
       <groupId>org.springframework.cloud</groupId>
       <artifactId>spring-cloud-dependencies</artifactId>
       <version>${spring-cloud.version}</version>
       <type>pom</type>
       <scope>import</scope>
     </dependency>
  </dependencies>
```

```
    </dependencyManagement>

    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
      </plugins>
    </build>
  </project>
```

❶ Spring Boot version

❷ Spring Cloud version to use

❸ Spring Cloud projects and other dependencies needed to run the ConfigServer

❹ Spring Cloud BOM (Bill of Materials) definition

NOTE All the pom.xml files should contain Docker dependencies and configurations, but in order to save space, we won't add those lines to the code listings. If you want to take a look at the Docker configuration for the configuration server, visit the chapter 5 folder in the GitHub repository: **https://github.com/ihuaylupo/manning-smia/tree/master/chapter5**.

We won't go through the entire pom file in detail, but we'll note a few key areas to begin with. In the Maven file in listing 5.1, we can see four important parts. The first is the Spring Boot version, and the next is the Spring Cloud version that we are going to use. In this example, we use version Hoxton.SR1 of Spring Cloud. The third point highlighted in the listing is the specific dependencies that we'll use in the service, and the last point is the Spring Cloud Config parent BOM (Bill of Materials) that we'll use.

This parent BOM contains all the third-party libraries and dependencies that are used in the cloud project and the version numbers of the individual projects that make up that version. In this example, we use the version defined previously in the `<properties>` section of the pom file. By using a BOM definition, we can guarantee that we'll use compatible versions of the subprojects in Spring Cloud. It also means that we don't have to declare version numbers for our subdependencies.

Come on, ride the train, the release train

Spring Cloud uses a non-traditional mechanism for labeling Maven projects. Because Spring Cloud is a collection of independent subprojects, the Spring Cloud team releases updates to the projects through what they call the "release train." All the subprojects that make up Spring Cloud are packaged under one Maven BOM and released as a whole.

The Spring Cloud team uses the names of London subway stops as the names of their releases, with each major release given a London subway stop that has the next highest letter. There have been several releases, from Angel, Brixton, Camden, Dalston, Edgware, Finchley, and Greenwich to Hoxton. Hoxton is the newest release, but it still has multiple release candidate branches for the subprojects within it.

One thing to note is that Spring Boot is released independently of the Spring Cloud release train. Therefore, different versions of Spring Boot are incompatible with different releases of Spring Cloud. You can see the version dependences between Spring Boot and Spring Cloud, along with the different subproject versions contained within the release train, by referring to the Spring Cloud website (**http://projects.spring.io/spring-cloud/**).

The next step in creating our Spring Cloud Config Server is to set up one more file to define the core configuration of the server so that it can run. This can be one of the following: application.properties, application.yml, bootstrap.properties, or bootstrap.yml.

The bootstrap file is a specific Spring Cloud file type and is loaded before the application.properties or application.yml files. The bootstrap file is used for specifying the Spring application name, the Spring Cloud Configuration Git location, encryption/decryption information, and so forth. Specifically, the bootstrap file is loaded by a parent Spring `ApplicationContext`, and that parent is loaded before the one that uses the application properties or YAML files.

As for the file extensions, .yml and .properties are just different data formats. You can choose the one you prefer. In this book, you'll see that we will use bootstrap.yml to define the configuration of the Config Server and the microservices.

Now, to continue, let's create our bootstrap.yml file in the /src/main/resources folder. This file tells the Spring Cloud Config service what port to listen in on, the application name, the application profiles, and the location where we will store the configuration data. Your bootstrap file should look like that shown in the following listing.

Listing 5.2 Creating our bootstrap.yml file

```
spring:
  application:
    name: config-server    ❶
server:
  port: 8071               ❷
```

❶ The Config Server application name (in this case, config-server)

❷ The server port

There are two important parts to highlight in listing 5.2. The first one is the application name. It is vital to name all of the services that we are going to create in our architecture for service discovery, which we will describe in the next chapter. The second point to note is the port the Spring Configuration Server is going to listen in on, in order to provide the requested configuration data.

2.1 Setting up the Spring Cloud Config bootstrap class

The next step in creating our Spring Cloud Config service is to set up the Spring Cloud Config bootstrap class. Every Spring Cloud service needs a bootstrap class that we can use to launch the service, as we explained in chapters 2 and 3 (where we created the licensing service).

Remember, this class contains several important parts: a Java `main()` method that acts as the entry point for the service to start in and a set of Spring annotations that tells the starting service what kind of behaviors Spring is going to launch for the service. The following listing shows the bootstrap class for our Spring Cloud Config Server.

Listing 5.3 Setting up the bootstrap class

```
package com.optimagrowth.configserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication                                    ❶

@EnableConfigServer                                       ❷

public class ConfigurationServerApplication {
```

```
  public static void main(String[] args) {                    ❸
   SpringApplication.run(ConfigurationServerApplication.class, args);
  }
}
```

❶ Our Config service is a Spring Boot application, so we must mark it with the @SpringBootApplication annotation.

❷ Enables the service as a Spring Cloud Config service

❸ The main method launches the service and starts the Spring container.

The next step is to define the search location for our configuration data. Let's start with the simplest example: the filesystem.

2.2 Using the Spring Cloud Config Server with a filesystem

The Spring Cloud Configuration Server uses an entry in the bootstrap.yml file to point to the repository that holds the application's configuration data. Setting up a filesystem-based repository is the easiest way to accomplish this. To do this, let's update our bootstrap file. The following listing shows the required contents for this file in order to set up a filesystem repository.

Listing 5.4 Our bootstrap.yml with a filesystem repository

```
spring:
  application:
    name: config-server
  profiles:
    active: native                                  ❶

  cloud:
    config:
      server:
      #Local configuration: This locations can either of
      classpath or locations in the filesystem.
        native:
        #Reads from a specific filesystem folder
          search-locations:
            file:///{FILE_PATH}                     ❷
server:
  port: 8071
```

❶ Sets the Spring profile associated with the backend repository (filesystem)

❷ Sets the search location where the configuration files are stored

Because we will use the filesystem for storing application configuration information, we must tell the Spring Cloud Config Server to run with the native profile. Remember, a Spring profile is a core feature that the Spring framework offers. It allows us to map our beans to different environments, such as dev, test, staging, production, native, and others.

NOTE Remember, `native` is just a profile created for the Spring Cloud Configuration Server, which indicates that the configuration files are going to be retrieved or read from the classpath or filesystem.

When we use a filesystem-based repository, we'll also use the native profile because it is a profile in the Config Server that doesn't use any Git or Vault configurations. Instead, it loads the configuration data directly from a local classpath or a filesystem. Finally, the last part of the bootstrap.yml shown in listing 5.4 provides the Spring Cloud configuration within the directory where the application data resides. For example:

```
server:
  native:
    search-locations: file:///Users/illary.huaylupo
```

The important parameter in the configuration entry is `search-locations`. This parameter provides a comma-separated list of the directories for each application that will have properties managed by the Config Server. In the previous example, we used a filesystem location (`file:///Users/illary.huaylupo`), but we can also indicate a specific classpath to look for our configuration data. This is set with the following code:

```
server:
  native:
    search-locations: classpath:/config
```

NOTE The `classpath` attribute causes Spring Cloud Config Server to look in the src/main/resources/config folder.

Now that we have set up our Spring Configuration Server, let's create our licensing service properties files. In order to make this example simple, we'll use the classpath search

location set in the preceding code snippet. Then, like the previous example, we'll create the licensing properties files in a /config folder.

2.3 Setting up the configuration files for a service

In this section, we will use the licensing service example that we began in the initial chapters of this book. It will serve as an example of how to use Spring Cloud Config.

NOTE In case you didn't follow along in the previous chapter's code listings, you can download the code created in chapter 4 from the following link: https://github.com/ihuaylupo/manning-smia/tree/master/chapter4.

Again, to keep this example simple, we will set up application configuration data for three environments: a default environment for when we run the service locally, a dev environment, and a production environment.

With Spring Cloud Config, everything works off a hierarchy. Your application configuration is represented by the name of the application and then a property file for each environment you want to configure. In each of these environments, we'll set up the following configuration properties:

- An example property that will be used directly by our licensing service
- A Spring Actuator configuration that we will use in the licensing service
- A database configuration for the licensing service

Figure 5.5 illustrates how we will set up and use the Spring Cloud Config service. One important point to mention is that as you build your Config service, it will be another microservice running in your environment. Once it's set up, the contents of the service can be accessed via an HTTP-based REST endpoint.

Figure 5.5 Spring Cloud Config exposes environment-specific properties as HTTP-based endpoints.

The naming convention for the application configuration files are *appname*-env.properties or *appname-env*.yml. As you can see from figure 5.5, the environment names translate directly into the URLs that will be accessed to browse the configuration information. Later, when we start the licensing microservice example, the environment we want to run the service against is specified by the Spring Boot profile that you pass in on the command line at service startup. If a profile isn't passed in on the command line,

Spring Boot defaults to the configuration data contained in the application.properties file packaged with the application.

Here is an example of some of the application configuration data we will serve up for the licensing service. This is the data that will be contained within the configserver/src/main/ resources/config/ licensing-service.properties file that was referred to in figure 5.5. Here is part of the contents of that file:

```
...
example.property= I AM THE DEFAULT
spring.jpa.hibernate.ddl-auto=none
spring.jpa.database=POSTGRESQL
spring.datasource.platform=postgres
spring.jpa.show-sql = true
spring.jpa.hibernate.naming-strategy = org.hibernate.cfg.ImprovedNamingStrategy
spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.PostgreSQLDialect
spring.database.driverClassName= org.postgresql.Driver
spring.datasource.testWhileIdle = true
spring.datasource.validationQuery = SELECT 1
management.endpoints.web.exposure.include=*
management.endpoints.enabled-by-default=true
```

Think before you implement

We advise against using a filesystem-based solution for medium to large cloud applications. Using the filesystem approach means that you need to implement a shared file mount point for all Configuration Servers that want to access the application's configuration data. Setting up shared filesystem servers in the cloud is doable, but it puts the onus of maintaining this environment on you.

We're showing the filesystem approach as the easiest example to use when getting your feet wet with the Spring Cloud Configuration Server. In a later section, we'll show you how to configure the Config Server to use GitHub and HashiCorp Vault to store your application configuration.

To continue, let's create a licensing-service-dev.properties file that contains only the development data. The dev properties file should contain the following parameters:

```
example.property= I AM DEV
spring.datasource.url = jdbc:postgresql://localhost:5432/ostock_dev
```

```
spring.datasource.username = postgres
spring.datasource.password = postgres
```

Now that we have enough work done to start the Configuration Server, let's go ahead and do that by using the `mvn spring-boot:run` command or the `docker-compose up` command.

NOTE From this point on, you will find a README file in the folder repository for each chapter. This file contains a section called "How to Use." This section describes how to run all the services together using the `docker-compose` command.

The server should now come up with the Spring Boot splash screen on the command line. If you point your browser to `http://localhost:8071/licensing-service/ default`, you'll see a JSON payload returned with all of the properties contained within the licensing-service.properties file. Figure 5.6 shows the results of calling this endpoint.

Figure 5.6 Spring Cloud Config exposes environment-specific properties as HTTP-based endpoints.

If you want to see the configuration information for the dev-based licensing service environment, select the GET `http://localhost:8071/licensing-service/dev` endpoint. Figure 5.7 shows the result of calling this endpoint.

NOTE The port is the one we set previously on the bootstrap.yml file.

Figure 5.7 Retrieving configuration information for the licensing service using the dev profile

If we look closely, we will see that when we selected the `dev` endpoint, the Spring Cloud Configuration Server returned both the default configuration properties and the dev licensing service configuration. The reason why Spring Cloud Config returns both sets of configuration information is that the Spring framework implements a hierarchical mechanism for resolving problems. When the Spring framework does this, it looks for the property defined in the default properties file first and then overrides the default with an environment-specific value if one is present. In concrete terms, if you define a property in the licensing-service.properties file and don't define it in any of the other en-

vironment configurations (for example, licensing-service-dev.properties), the Spring framework will use the default value.

NOTE This isn't the behavior you'll see by directly calling the Spring Cloud Config `REST` endpoint. The `REST` endpoint returns all configuration values for both the default and environment-specific value that was called.

Now that we finished configuring everything in our Spring Cloud Config service, let's move on to integrating Spring Cloud Config with our licensing microservice.

5.3 Integrating Spring Cloud Config with a Spring Boot client

In the previous chapters, we built a simple skeleton of our licensing service that did nothing more than to return a hardcoded Java object representing a single licensing record. In this section, we will build the licensing service with a PostgreSQL database to hold the licensing data.

Why use PostgreSQL?

PostgreSQL, also known as Postgres, is considered an enterprise system and one of the most interesting and advanced options for open source relational database management systems (RDBMSs). PostgreSQL has many advantages compared to other relational databases, but the main ones are that it offers a single license that is entirely free and open for anyone to use. The second advantage is that, in terms of its ability and functionality, it allows us to work with more significant amounts of data without increasing the complexity of the queries. Here are some of the main features of Postgres:

- Postgres uses a multiversion concurrency control that adds an image of the database status to each transaction, which produces consistent transactions with better performance advantages.
- Postgres doesn't use reading locks when it executes a transaction.
- Postgres has something called hot standby, which allows the client to search in the servers while the server is in recovery or standby mode. In other words, Postgres performs maintenance without completely locking down the database.

Some of the main characteristics of PostgreSQL are as follows:

- It is supported by languages such as C, C++, Java, PHP, Python, and more.
- It is capable of serving many clients while delivering the same information from its tables without blockages.

- It supports working with views so users can query the data differently from how it is stored.
- It is an object-relational database, allowing us to work with data as if it were objects, thus offering object-oriented mechanisms.
- It allows us to store and query JSON as a data type.

We are going to communicate with the database using Spring Data and map our data from the licensing table to a Plain Old Java Object (POJO) holding the data. We'll use the Spring Cloud Configuration Server to read the database connection and a simple property. Figure 5.8 shows what's going to happen between the licensing service and the Spring Cloud Config service.

When the licensing service is first started, we'll pass it three pieces of information: the Spring profile, the application name, and the endpoint the licensing service should use to communicate with the Spring Cloud Config service. The Spring profile maps to the environment of the properties being retrieved for the Spring service.

Figure 5.8 Retrieving configuration information using the `dev` profile

When the licensing service boots up, it will contact the Spring Cloud Config service via an endpoint built from the Spring profile passed into it. The Spring Cloud Config service will then use the configured backend repository (filesystem, Git, or Vault) to retrieve the configuration information specific to the Spring profile value passed in on the URI. The appropriate property values are then passed back to the licensing service. The Spring Boot framework will then inject these values into the appropriate parts of the application.

3.1 Setting up the licensing service Spring Cloud Config Service dependencies

Let's change our focus from the Spring Cloud Configuration Server to the licensing service. The first thing you need to do is to add a couple more entries to the Maven file in your licensing service. The following listing gives the entries that we need to add.

Listing 5.5 Adding Maven dependencies to the licensing service

```
//Parts of pom.xml omitted for conciseness
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>
```

```
        spring-cloud-starter-config
    </artifactId>                          ❶
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>
        spring-boot-starter-data-jpa
    </artifactId>                          ❷
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>                ❸
    <artifactId>postgresql</artifactId>
</dependency>
```

❶ Tells Spring Boot to pull down the dependencies needed for Spring Cloud Config

❷ Tells Spring Boot to use the Java Persistence API (JPA) in your service

❸ Tells Spring Boot to pull down the Postgres drivers

The first dependency's artifact ID, `spring-cloud-starter-config`, contains all the classes needed to interact with the Spring Cloud Config Server. The second and third dependencies, `spring-boot-starter-data-jpa` and `postgresql`, import the Spring Data Java Persistence API (JPA) and the Postgres JDBC drivers.

3.2 Configuring the licensing service to use Spring Cloud Config

After the Maven dependencies have been defined, we need to tell the licensing service where to contact the Spring Cloud Configuration Server. In a Spring Boot service that uses Spring Cloud Config, configuration information can be set in one of these files: bootstrap.yml, bootstrap.properties, application.yml, or application.properties.

As we mentioned previously, the bootstrap.yml file reads the application properties before any other configuration information. In general, the bootstrap.yml file contains the application name for the service, the application profile, and the URI to connect to a Configuration Server. Any other configuration information that you want to keep local to the service (and not stored in Spring Cloud Config) can be set in the services in the local application.yml file.

Usually, the information you store in the application.yml file is configuration data that you might want to have available to a service even if the Spring Cloud Config service is

unavailable. Both the bootstrap.yml and application.yml files are stored in a project's src/main/resources directory.

To have the licensing service communicate with your Spring Cloud Config service, these parameters can be defined in the bootstrap.yml file, in the docker-compose.yml file of the licensing service, or via JVM arguments when you start the service. The following listing shows how bootstrap.yml should look in your application if you choose this option.

Listing 5.6 Configuring the licensing service's bootstrap.yml

```
spring:
  application:
    name: licensing-service        ❶
  profiles:
    active: dev                    ❷
  cloud:
    config:
      uri: http://localhost:8071   ❸
```

❶ Specifies the name of the licensing service so that the Spring Cloud Config client knows which service is being looked up

❷ Specifies the default profile the service should run. The profile maps to an environment.

❸ Specifies the location of the Spring Cloud Config Server

NOTE The Spring Boot application supports two mechanisms to define a property: YAML (YAML Ain't Markup Language) and a dot-separated property name. We will choose YAML as the means for configuring our application. The hierarchical format of YAML property values maps directly to these names: `spring.application.name`, `spring.profiles.active`, and `spring .cloud.config.uri`.

The `spring.application.name` is the name of your application (for example, licensing service) and *must* map directly to the name of the config directory within your Spring Cloud Configuration Server. The second property, `spring.profiles.active`, tells Spring Boot which profile the application should run as. Remember, a *profile* is a mechanism to differentiate the configuration data consumed by the Spring Boot application. For the licensing service's profile, you'll support the environment the service is going to map directly to in your cloud configuration environment. For instance, by passing in `dev` as your pro-

file, the Config Server will use the `dev` properties. If you don't set a profile, the licensing service will use the default profile.

The third and last property, `spring.cloud.config.uri`, is the location where the licensing service will look for the Config Server endpoint. In this example, the licensing service looks for the configuration server at `http://localhost:8071`.

Later in this chapter, you'll see how to override the different properties defined in the bootstrap.yml and application.yml files on application startup. This allows you to tell the licensing microservice which environment it should be running in. Now, if you bring up the Spring Cloud Config service with the corresponding Postgres database running on your local machine, you can launch the licensing service using its `dev` profile. This is done by changing to the licensing service's directory and issuing the following command:

```
mvn spring-boot:run
```

NOTE You first need to launch the Configuration Server to retrieve the configuration data for the licensing service.

By running this command without any properties set, the licensing server automatically attempts to connect to the Spring Cloud Configuration Server using the endpoint (in this case, our endpoint is `http://localhost:8071`) and the active profile (`dev`) defined previously in the bootstrap.yml file of the licensing service.

If you want to override these default values and point to another environment, you can do so by compiling the licensing service project down to a JAR file, and then run the JAR with a `D` system property override. The following command-line call demonstrates how to launch the licensing service, passing all the commands via JVM arguments:

```
java  -Dspring.cloud.config.uri=http://localhost:8071 \
    -Dspring.profiles.active=dev \
    -jar target/licensing-service-0.0.1-SNAPSHOT.jar
```

This example demonstrates how to override Spring properties via the command line. With this command line, we override these two parameters:

```
spring.cloud.config.uri
spring.profiles.active
```

NOTE If you try to run the licensing service that you downloaded from the GitHub repository (https://github.com/ihuaylupo/manning-smia/tree/master/chapter5) from your desktop using the previous Java command, it will fail because of two reasons. The first one is that you don't have a desktop Postgres server running, and the second is because the source code in the GitHub repository uses encryption on the Config Server. We'll cover using encryption later in this chapter.

In the examples, we hardcoded the values to pass in to the `-D` parameter values. In the cloud, most of the application configuration data you'll need will be in your configuration server.

All the code examples for each chapter can be completely run from within Docker containers. With Docker, you simulate different environments through environment-specific Docker Compose files, which orchestrate the startup of all of your services. Environment-specific values needed by the containers are passed in as environment variables to the container. For example, to start your licensing service in a dev environment, the dev/docker-compose.yml file contains the entry displayed in the next listing for the licensing service.

Listing 5.7 Dev docker-compose.yml

```
licensingservice:
  image: ostock/licensing-service:0.0.1-SNAPSHOT
  ports:
    - "8080:8080"

  environment:                    (1)
    SPRING_PROFILES_ACTIVE: "dev"    (2)

    SPRING_CLOUD_CONFIG_URI:
      http://configserver:8071    (3)
```

(1) Specifies the start of the environment variables for the licensing service container

(2) Passes the SPRING_PROFILES_ACTIVE environment variable to the Spring Boot service command line and tells Spring Boot what profile should be run

(3) The endpoint of the Config service

The environment entry in the YML file contains the values of two variables: `SPRING_PROFILES_ACTIVE`, which is the Spring Boot profile the licensing service will run under,

and `SPRING_CLOUD_CONFIG_URI`, which is passed to your licensing service and defines the Spring Cloud Configuration Server instance where the service will read its configuration data. Once you have Docker Compose file set up, you can run the services just by executing the following command where the Docker Compose file is located:

```
docker-compose up
```

Because you enhance all your services with introspection capabilities via Spring Boot Actuator, you can confirm the environment you are running in by selecting the following endpoint: `http://localhost:8080/actuator/env`. The `/env` endpoint provides a complete list of the configuration information about the service, including the properties and endpoints the service is booted with (figure 5.9).

Figure 5.9 You can check the licensing configuration service by calling the `/actuator/env` endpoint. In the code, you can see how both the `licensing-service.properties` and the `licensing-service-dev.properties` are displayed.

On exposing too much information

Every organization is going to have different rules about how to implement security around their services. Many organizations believe services shouldn't broadcast any information about themselves and won't allow things like a `/env` endpoint to be active on a service. Their belief (rightfully so) is that this provides too much information for a potential hacker.

Spring Boot provides a wealth of capabilities on how to configure what information is returned by the Spring Actuator endpoints. That's outside the scope of this book, however. Craig Walls' excellent book, Spring Boot in Action (Manning, 2016), covers this subject in detail. We highly recommend that you review your corporate security policies and Walls' book to provide the right level of information you want to expose through Spring Actuator.

3.3 Wiring in a data source using Spring Cloud Config Server

At this point, the database configuration information is directly injected into your microservice. With the database configuration set, configuring your licensing microservice becomes an exercise in using standard Spring components to build and retrieve the data from the Postgres database. In order to continue with the example, we need to refactor

the licensing into different classes, where each class has separate responsibilities. These classes are shown in table 5.2.

Table 5.2 Licensing service classes and locations

Class name	Location
License	com.optimagrowth.license.model
LicenseRepository	com.optimagrowth.license.repository
LicenseService	com.optimagrowth.license.service

The `License` class is the model class that will hold the data retrieved from your licensing database. The following listing shows the code for this class.

Listing 5.8 The JPA model code for a single license record

```
package com.optimagrowth.license.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

import org.springframework.hateoas.RepresentationModel;

import lombok.Getter;
import lombok.Setter;
import lombok.ToString;

@Getter @Setter @ToString
@Entity                                    (1)
@Table(name="licenses")                    (2)
public class License {

  @Id                                      (3)
  @Column(name = "license_id", nullable = false)     (4)
  private String licenseId;
  private String description;
  @Column(name = "organization_id", nullable = false)
  private String organizationId;
  @Column(name = "product_name", nullable = false)
```

```
    private String productName;
    @Column(name = "license_type", nullable = false)
    private String licenseType;
    @Column(name="comment")
    private String comment;

    public License withComment(String comment){
      this.setComment(comment);
      return this;
    }
}
```

❶ Tells Spring that this is a JPA class

❷ Maps to the database table

❸ Marks this field as a primary key

❹ Maps the field to a specific database column

In the listing, the `License` class uses several JPA annotations that will help the Spring Data framework map the data from the licenses table in the Postgres database to a Java object. The `@Entity` annotation lets Spring know that this Java POJO is going to be mapping objects that will hold data. The `@Table` annotation tells Spring/JPA what database table to map. The `@Id` annotation identifies the primary key for the database. Finally, each one of the columns from the database that will be mapped to individual properties is marked with a `@Column` attribute.

TIP If your attribute has the same name as the database column, you don't need to add the `@Column` annotation.

The Spring Data and JPA framework provides your basic CRUD methods (Create, Replace, Update, Delete) for accessing a database. Table 5.3 shows some of these.

Table 5.3 CRUD methods for Spring Data and the JPA framework

Method	Description
count()	Returns the number of entities available
delete(entity)	Deletes a given entity
deleteAll()	Deletes all entities managed by the repository
deleteAll(entities)	Deletes the given entities
deleteById(id)	Deletes the entity with the given ID
existsById(id)	Returns whether an entity with the given ID exists
findAll()	Returns all instances of the type
findAllById(ids)	Returns all instances of a given type `T` with the given IDs
findById(ID id)	Retrieves an entity by its ID
save(entity)	Saves a given entity
saveAll(entities)	Saves all given entities

If you want to build methods beyond this, you can use a Spring Data Repository interface and basic naming conventions to build those methods. At startup, Spring will parse the name of the methods from the Repository interface, convert them to an SQL statement based on the names, and then generate a dynamic proxy class (under the cover) to do the work. The repository for the licensing service is shown in the following listing.

Listing 5.9 The `LicenseRepository` interface defining the query methods

```
package com.optimagrowth.license.repository;

import java.util.List;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;
import com.optimagrowth.license.model.License;
```

```
@Repository                                    ❶

public interface LicenseRepository
    extends CrudRepository<License,String> {    ❷
  public List<License> findByOrganizationId
                (String organizationId);        ❸
  public License findByOrganizationIdAndLicenseId
                (String organizationId,
                 String licenseId);
}
```

❶ Tells Spring Boot that this is a JPA repository class. Annotation is optional when we extend from a CrudRepository.

❷ Extends the Spring CrudRepository

❸ Parses query methods into a SELECT...FROM query

The repository interface, `LicenseRepository`, is marked with `@Repository`, which tells Spring that it should treat this interface as a repository and generate a dynamic proxy for it. The dynamic proxy, in this case, provides a set of fully featured, ready-to-use objects.

Spring offers different types of repositories for data access. In this example, we use the Spring `CrudRepository` base class to extend our `LicenseRepository` class. The `CrudRepository` base class contains basic CRUD methods. In addition to the CRUD methods extended from `CrudRepository`, we added two custom query methods to the `LicenseRepository` interface for retrieving data from the licensing table. The Spring Data framework pulls apart the name of the methods to build a query to access the underlying data.

NOTE The Spring Data framework provides an abstraction layer over various database platforms and isn't limited to relational databases. NoSQL databases, such as MongoDB and Cassandra, are also supported.

Unlike the previous incarnation of the licensing service in chapter 3, you've now separated the business and data access logic for the licensing service out of the `LicenseController` and into a standalone `Service` class called `LicenseService`. Listing 5.10 shows the license service. Between this `LicenseService` class and the versions seen in the previous chapters, there are a lot of changes because we added the database connection. Feel free to download the file from the following link:

https://github.com/ihuaylupo/manning-smia/tree/master/chapter5/licensing-service/src/main/java/com/optimagrowth/license/service/LicenseService.java

Listing 5.10 A `LicenseService` class to execute database commands

```
@Service
public class LicenseService {

  @Autowired
  MessageSource messages;
  @Autowired
  private LicenseRepository licenseRepository;
  @Autowired
  ServiceConfig config;

  public License getLicense(String licenseId, String organizationId){
     License license = licenseRepository
       .findByOrganizationIdAndLicenseId(organizationId, licenseId);
     if (null == license) {
        throw new IllegalArgumentException(
           String.format(messages.getMessage(
             "license.search.error.message", null, null),
              licenseId, organizationId));
     }
     return license.withComment(config.getProperty());
  }

  public License createLicense(License license){
     license.setLicenseId(UUID.randomUUID().toString());
     licenseRepository.save(license);
     return license.withComment(config.getProperty());
  }

  public License updateLicense(License license){
     licenseRepository.save(license);
     return license.withComment(config.getProperty());
  }

  public String deleteLicense(String licenseId){
     String responseMessage = null;
     License license = new License();
     license.setLicenseId(licenseId);
     licenseRepository.delete(license);
     responseMessage = String.format(messages.getMessage(
          "license.delete.message", null, null),licenseId);
```

```
        return responseMessage;
    }
}
```

The controller, service, and repository classes are wired together using the standard Spring `@Autowired` annotation. Next, let's look at reading the configuration properties in the `LicenseService` class.

3.4 Directly reading properties using @ConfigurationProperties

In the `LicenseService` class, you might have noticed that we set the `license.withComment()` value in the `getLicense()` method with a value from the `config.getProperty()` class. The code referred to is shown here:

```
return license.withComment(config.getProperty());
```

If you look at the `com.optimagrowth.license.config.ServiceConfig.java` class, you'll see that the class is annotated with the following. (Listing 5.11 shows the `@ConfigurationProperties` annotation that we'll use.)

```
@ConfigurationProperties(prefix= "example")
```

Listing 5.11 Using `ServiceConfig` to centralize application properties

```
package com.optimagrowth.license.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@ConfigurationProperties(prefix = "example")
public class ServiceConfig{

  private String property;

  public String getProperty(){
    return property;
  }
}
```

While Spring Data "auto-magically" injects the configuration data for the database into a database connection object, all other custom properties can be injected using the `@ConfigurationProperties` annotation. With the previous example,

```
@ConfigurationProperties(prefix= "example")
```

pulls all the `example` properties from the Spring Cloud Configuration Server and injects these into the `property` attribute on the `ServiceConfig` class.

TIP While it's possible to directly inject configuration values into properties in individual classes, we've found it useful to centralize all of the configuration information into a single configuration class and then inject the configuration class into where it's needed.

3.5 Refreshing your properties using Spring Cloud Config Server

One of the first questions that comes up from development teams when they want to use the Spring Cloud Configuration Server is how can they dynamically refresh their applications when a property changes. Rest assured. The Config Server always serves the latest version of a property. Changes made to a property via its underlying repository will be up to date!

Spring Boot applications, however, only read their properties at startup, so property changes made in the Config Server won't be automatically picked up by the Spring Boot application. But Spring Boot Actuator offers a `@RefreshScope` annotation that allows a development team to access a `/refresh` endpoint that will force the Spring Boot application to reread its application configuration. The following listing shows this annotation in action.

Listing 5.12 The `@RefreshScope` annotation

```
package com.optimagrowth.license;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.context.config.annotation.RefreshScope;

@SpringBootApplication
@RefreshScope
public class LicenseServiceApplication {

  public static void main(String[] args) {
```

```
    SpringApplication.run(LicenseServiceApplication.class, args);
  }

}
```

Note a couple of things about the `@RefreshScope` annotation. This annotation only reloads the custom Spring properties you have in your application configuration. Items like your database configuration used by Spring Data won't be reloaded by this annotation.

On refreshing microservices

When using Spring Cloud Config service with microservices, one thing you need to consider before you dynamically change properties is that you might have multiple instances of the same service running. You'll need to refresh all of those services with their new application configurations. There are several ways you can approach this problem.

Spring Cloud Config service offers a push-based mechanism called Spring Cloud Bus that allows the Spring Cloud Configuration Server to publish to all the clients using the service where a change occurs. Spring Cloud Bus requires an extra piece of running middleware: RabbitMQ. This is an extremely useful means of detecting changes, but not all Spring Cloud Config backends support the push mechanism (the Consul server, for example). In the next chapter, you'll use Spring Cloud service discovery and Eureka to register all instances of a service.

One technique that we've used to handle application configuration refresh events is to refresh the application properties in Spring Cloud Config. Then we write a simple script to query the service discovery engine to find all instances of a service and call the `/refresh` endpoint directly.

You can also restart all the servers or containers to pick up the new property. This is a trivial exercise, especially if you're running your services in a container service such as Docker. Restarting Docker containers literally takes seconds and will force a reread of the application configuration.

Remember that cloud-based servers are ephemeral. Don't be afraid to start new instances of a service with new configurations, then direct traffic to the new services and tear down the old ones.

3.6 Using Spring Cloud Configuration Server with Git

As mentioned earlier, using a filesystem as the backend repository for the Spring Cloud Configuration Server can be impractical for a cloud-based application. That's because the development team has to set up and manage a shared filesystem that's mounted on all instances of the Config Server, and the Config Server integrates with different backend repositories that can be used to host the application configuration properties.

One approach that we've used successfully is to employ a Spring Cloud Configuration Server with a Git source control repository. By using Git, you can get all the benefits of putting your configuration management properties under source control and provide an easy mechanism to integrate the deployment of your property configuration files in your build and deployment pipeline. To use Git, we need to add the configuration to the Spring Cloud Config service bootstrap.yml file. The next listing shows how.

Listing 5.13 Adding Git support to the Spring Cloud bootstrap.yml

```
spring:
  application:
    name: config-server
  profiles:
    active:
     - native, git                          (1)
  cloud:
    config:
      server:
        native:
          search-locations: classpath:/config
        git:                                (2)
          uri: https://github.com/ihuaylupo/
               config.git                   (3)
          searchPaths: licensingservice     (4)
server:
  port: 8071
```

(1) Maps all the profiles (this is a comma-separated list)

(2) Tells Spring Cloud Config to use Git as a backend repository

(3) Tells Spring Cloud Config the URL to the Git server and repo

(4) Tells Spring Cloud Config what path in Git to use to look for the config files

The four key pieces of configuration properties in the previous listing include the following:

- `spring.profiles.active`
- `spring.cloud.config.server.git`
- `spring.cloud.config.server.git.uri`
- `spring.cloud.config.server.git.searchPaths`

The `spring.profiles.active` property sets all the active profiles for the Spring Config service. This comma-separated list uses the same precedence rules as a Spring Boot application: active profiles have precedence over default profiles, and the last profile is the winner. The `spring.cloud.config.server.git` property tells the Spring Cloud Config Server to use a non-filesystem-based backend repository. In the previous listing, we connected to the cloud-based Git repository, GitHub.

NOTE If you're authorized to use GitHub, you need to set the username and password (personal token or SSH configuration) in the Git configuration on the bootstrap.yml of the configuration server.

The `spring.cloud.config.server.git.uri` property provides the URL of the repository you're connecting to. And finally, the `spring.cloud.config.server.git .searchPaths` property tells the Config Server the relative path on the Git repository that will be searched when the Cloud Config Server boots up. Like the filesystem version of the configuration, the value in the `spring.cloud.config.server.git.searchPaths` attribute will be a comma-separated list for each service hosted by the configuration service.

NOTE The default implementation of an environment repository in Spring Cloud Config is the Git backend.

3.7 Integrating Vault with the Spring Cloud Config service

As mentioned earlier, there is another backend repository that we will use: the HashiCorp Vault. Vault is a tool that allows us to securely access secrets. We can define secrets as any piece of information we want to restrict or control access to, such as passwords, certificates, API keys, and so forth.

To configure Vault in our Spring Config service, we must add a Vault profile. This profile enables integration with Vault and allows us to securely store the application properties of our microservices. To achieve this integration, we will use Docker to create a Vault container with the following command:

```
docker run -d -p 8200:8200 --name vault -e 'VAULT_DEV_ROOT_TOKEN_ID=myroot'
↪ -e 'VAULT_DEV_LISTEN_ADDRESS=0.0.0.0:8200' vault
```

The `docker run` command contains these parameters:

- `VAULT_DEV_ROOT_TOKEN_ID` —This parameter sets the ID of the generated root token. The *root token* is the initial access token to start configuring Vault. This sets the ID of the initial generated root token to a given value.
- `VAULT_DEV_LISTEN_ADDRESS` —This parameter sets the IP address and port of the development server listener; the default value is 0.0.0.0:8200.

NOTE In this example, we will run Vault locally. If you need additional info on how to run Vault in server mode, we highly recommend that you visit the official Vault Docker image information at **https://hub.docker.com/_/vault**.

Once the latest Vault image is pulled into Docker, we can start creating our secrets. To make this example more straightforward, we will use the Vault UI, but if you prefer to move on with the CLI commands, go for it.

3.8 Vault UI

Vault offers a unified interface that facilitates the process of creating secrets. To access this UI, we need to enter the following URL: `http://0.0.0.0:8200/ui/vault/auth`. This URL was defined by the `VAULT_DEV_LISTEN_ADDRESS` parameter set with the `docker run` command. Figure 5.10 shows the login page for the Vault UI.

Figure 5.10 Login page in the Vault UI. Enter the following URL in the Token field to sign in: `http://0.0.0.0:8200/ui/vault/auth`.

The next step is to create a secret. To create the secret after you've logged in, click the Secrets tab in the Vault UI dashboard. For this example, we will create a secret called `secret/licensingservice` with a property called `license.vault.property` and set its value to `Welcome to vault`. Remember, access to this piece of information will be restricted and will be encrypted. To achieve this, first we need to create a new secret engine and then add the specific secret to that engine. Figure 5.11 shows how to create this with the Vault UI.

Figure 5.11 Creating a new secret engine in the Vault UI

Now, that we have our new secret engine, let's create our secret. Figure 5.12 shows you how.

Figure 5.12 Creating a new secret in the Vault UI

Now that we have configured the Vault and a secret, let's configure our Spring Cloud Config Server to communicate with Vault. To do that, we'll add the Vault profile to our bootstrap.yml file for the Config Server. The next listing shows how your bootstrap file should look.

Listing 5.14 Adding Vault to the Spring Cloud bootstrap.yml

```
spring:
   application:
     name: config-server
   profiles:
     active:
     - vault
   cloud:
     config:
      server:
        vault:              (1)
          port: 8200        (2)
          host: 127.0.0.1   (3)
          kvVersion: 2      (4)
server:
  port: 8071
```

(1) Tells Spring Cloud Config to use Vault as the backend repository

(2) Tells Spring Cloud Config the Vault port

(3) Tells Spring Cloud Config the Vault host

(4) Sets the kv secrets engine version.

NOTE An important point here is the kv secrets engine version. The default value for `spring.cloud.config.server.kv-version` is `1`. But it is recommended to use version 2 when we use Vault 0.10.0 or later.

Now that we have everything set, let's test our Config Server via an HTTP request. Here you can use a cURL command or some REST client such as Postman:

```
$ curl -X "GET" "http://localhost:8071/licensing-service/default" -H
↪ "X-Config-Token: myroot"
```

If everything was configured successfully, the command should return a response like the following:

```
{
   "name": "licensing-service",
   "profiles": [
      "default"
   ],
   "label": null,
   "version": null,
   "state": null,
   "propertySources": [
      {
         "name": "vault:licensing-service",
         "source": {
            "license.vault.property": "Welcome to vault"
         }
      }
   ]
}
```

5.4 Protecting sensitive configuration information

By default, the Spring Cloud Configuration Server stores all properties in plain text within the application's configuration files. This includes sensitive information such as database credentials and so forth. It's extremely poor practice to keep sensitive credentials stored as plain text in your source code repository. Unfortunately, it happens far more often than you might think.

Spring Cloud Config gives you the ability to encrypt your sensitive properties easily. Spring Cloud Config supports using both symmetric (shared secret) and asymmetric en-

cryption (public/private) keys. Asymmetric encryption is more secure than symmetric encryption because it uses modern and more complex algorithms. But sometimes it is more convenient to use the symmetric key because we only need to define a single property value in the bootstrap.yml file in the Config Server.

4.1 Setting up a symmetric encryption key

The symmetric encryption key is nothing more than a shared secret that's used by the encrypter to encrypt a value and by the decrypter to decrypt a value. With the Spring Cloud Configuration Server, the symmetric encryption key is a string of characters you select that can be either set in the bootstrap.yml file of your Config Server or passed to the service via an OS environment variable, `ENCRYPT_KEY`. You can select the option that best suits your needs.

NOTE Your symmetric key should be 12 or more characters long and, ideally, should be a random set of characters.

Let's start by looking at an example of how to configure the symmetric key on the bootstrap file for the Spring Cloud Configuration Server. The following listing shows how to do this.

Listing 5.15 Setting a symmetric key in the boostrap.yml file

```
cloud:
   config:
     server:
       native:
         search-locations: classpath:/config
       git:
          uri: https://github.com/ihuaylupo/config.git
          searchPaths: licensingservice

server:
  port: 8071

encrypt:
  key: secretkey      (1)
```

(1) Tells the Config Server to use this value as the symmetric key

For the purposes of this book, we will always set the `ENCRYPT_KEY` environment variable to `export ENCRYPT_KEY=IMSYMMETRIC`. Feel free to use the bootstrap.yml file

property if you need to locally test without using Docker.

Managing encryption keys

In this book, we did two things that we wouldn't normally recommend in a production deployment:

- We set the encryption key to a phrase. We wanted to keep the key simple so that we could remember it and so that it would fit nicely when reading the text. In a real-world deployment, we use a separate encryption key for each environment we deploy to, and we use random characters as a key.
- We've hardcoded the `ENCRYPT_KEY` environment variable directly in the Docker files used for the book. We did this so that you as the reader can download the files and start them without having to remember to set an environment variable.

In a real run-time environment, we would reference the `ENCRYPT_KEY` as an OS environment variable inside a Dockerfile. Be aware of this and don't hardcode your encryption key inside your Dockerfiles. Remember, your Dockerfiles are supposed to be kept under source control.

4.2 Encrypting and decrypting a property

We are now ready to begin encrypting properties for use with Spring Cloud Config. We will encrypt the licensing service Postgres database password you use to access the O-stock data. This property, called `spring.datasource.password`, is currently set as plain text with `postgres` as its value.

When you fire up your Spring Cloud Config instance, Spring Cloud Config detects that the `ENCRYPT_KEY` environment variable or the bootstrap file property is set, and automatically adds two new endpoints, `/encrypt` and `/decrypt` to the Spring Cloud Config service. We will use the `/encrypt` endpoint to encrypt the `postgres` value. Figure 5.13 shows how to encrypt the `postgres` value using the `/encrypt` endpoint and Postman.

Figure 5.13 Encrypting the Spring data source password using the `/encrypt` endpoint

NOTE When you call the `/encrypt` or `/decrypt` endpoints, you need to make sure you do a POST to these endpoints.

If we wanted to decrypt the value, we'd need to use the `/decrypt` endpoint, passing in the encrypted string. We can now add the encrypted property to our GitHub or filesystem-based configuration file for the licensing service using the syntax shown in the following listing.

Listing 5.16 Adding an encrypted value to the licensing service properties file

```
spring.datasource.url = jdbc:postgresql://localhost:5432/ostock_dev
spring.datasource.username = postgres
spring.datasource.password = {cipher}
➥ 559ac661a1c93d52b9e093d3833a238a142de7772961d94751883b17c41746a6
```

The Spring Cloud Configuration Server requires all encrypted properties to be prepended with a value of `{cipher}`. This value tells the Config Server that it's dealing with an encrypted value.

5.5 Closing thoughts

Application configuration management might seem like a mundane topic, but it's of critical importance in a cloud-based environment. As we'll discuss in more detail in later chapters, it's vital that your applications and the servers these run on be immutable and that the entire server being promoted is never manually configured between environments. This flies in the face of traditional deployment models, where you deploy an application artifact (for example, a JAR or WAR file) along with its property files to a "fixed" environment.

With a cloud-based model, however, the application configuration data should be segregated completely from the application. The appropriate configuration data needs are then injected at run time so that the same server/application artifacts are consistently promoted throughout all environments.

mmary

- The Spring Cloud Configuration Server (aka Config Server) allows you to set up application properties with environment-specific values.
- Spring uses profiles to launch a service to determine what environment properties are to be retrieved from the Spring Cloud Config service.
- The Spring Cloud Config service can use a file-based, Git-based, or Vault-based application configuration repository to store application properties.

- The Spring Cloud Config service lets you encrypt sensitive property files using symmetric and asymmetric encryption.

Previous chapter
< 4 Welcome to Docker

Next chapter
6 On service discovery >